CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Objective 5: Subagent Tool Restrictions

> **Exam objective:** Configure subagent invocations with appropriate tool restrictions, context scoping, and system prompts that constrain each agent to its designated role.

Q1.

A coordinator delegates a read-only code review to a subagent. Which configuration best aligns with the objective?

A. Grant full edit, network, and deployment access in case the subagent finds something important B. Limit the subagent to read-only repository inspection tools and a review-focused system prompt C. Allow shell access only if the subagent promises not to change files D. Remove all tool restrictions so the subagent can work faster

Answer: B **Explanation:** Least-privilege delegation means the subagent should receive only the tools required for its role. A read-only reviewer does not need edit or deployment capability, and promises are weaker than enforced restrictions.

Q2.

Why are tool restrictions important even when a subagent is trusted?

A. They reduce the blast radius of mistakes and keep behavior aligned with assigned scope B. They guarantee higher model reasoning quality C. They eliminate the need for clear prompts D. They make context scoping unnecessary

Answer: A **Explanation:** Restrictions are an architectural control, not a judgment on trustworthiness. They limit accidental or out-of-scope actions, but they do not replace good prompting or improve reasoning by themselves.

Q3.

A subagent is asked to summarize benchmark results from a known CSV file. Which tool set is most appropriate?

A. File-read capability only, plus any parsing tool required for the CSV B. Full repository write access and network access C. Database admin credentials and deployment tools D. No tools at all, because summaries should rely on memory

Answer: A **Explanation:** The subagent only needs access to the data source and perhaps a parser to complete its assigned role. Anything broader violates least privilege without adding necessary capability.

Q4.

What is the purpose of context scoping in a subagent invocation?

A. To ensure the subagent sees only the relevant task material instead of the entire coordinator history B. To disable tool calls automatically C. To force the subagent to ask permission after every step D. To maximize token usage so details are never omitted

Answer: A **Explanation:** Context scoping keeps the subagent focused, reduces distraction, and limits exposure to irrelevant or sensitive information. It does not inherently disable tools or mandate per-step approvals.

Q5.

A coordinator sends a subagent the entire transcript of a multi-day project, even though the subagent only needs the latest diff and test failure. What is the main risk?

A. The subagent may be distracted by irrelevant context and operate less efficiently or less securely B. The subagent will always refuse the task C. The larger context guarantees better precision D. Tool restrictions become impossible to apply

Answer: A **Explanation:** Excess context can obscure the real task and expose information beyond what the subagent needs. Good scoping gives enough context for completion, but not everything available.

Q6.

Which system prompt best constrains a subagent to its designated role as a security reviewer?

A. "You may edit files if you think a fix is obvious." B. "Investigate only high-confidence exploitable security issues in the provided scope; do not modify files." C. "Be generally helpful with anything you notice." D. "Try to finish the whole project if time permits."

Answer: B **Explanation:** This system prompt defines the role, limits the scope of findings, and forbids modification, aligning the subagent with a focused security-review function. The other prompts allow drift into broader or write-capable behavior.

Q7.

A subagent is responsible only for log analysis, but the coordinator also grants source-code editing tools. Which statement is most accurate?

A. The extra tools are harmless if the prompt says not to use them B. The configuration is broader than necessary and violates least-privilege design C. Editing tools are required because logs may imply code changes D. Tool restrictions matter only for external network access

Answer: B **Explanation:** Least privilege is about enforcing the minimum needed capabilities, not merely discouraging use of unnecessary tools. If the role is log analysis, code editing should not be available by default.

Q8.

How do system prompts and tool restrictions complement each other?

A. System prompts define intended behavior, while tool restrictions enforce the capability boundary B. System prompts make tool restrictions redundant C. Tool restrictions replace the need to describe the task D. They should never be used together because they conflict

Answer: A **Explanation:** Prompts guide what the agent should do, and tool restrictions control what it can do. Using both together creates stronger alignment than either one alone.

Q9.

Which delegation is the best example of proper restriction and scoping?

A. A documentation subagent receives only the changed markdown files, a style summary, and file-read/edit tools limited to those docs B. A documentation subagent receives full production deployment rights in case examples need testing C. A testing subagent receives access to finance spreadsheets unrelated to the repository D. A metrics subagent receives the entire company knowledge base because it might be useful

Answer: A **Explanation:** The tools and context are tightly aligned to the assigned documentation role and limited to the relevant artifacts. The other options expose unrelated or overly powerful capabilities.

Q10.

What is the clearest reason to tailor tool restrictions per subagent instead of using one default tool set for all?

A. Different roles require different minimum capabilities, and a universal broad set increases unnecessary risk B. Tailoring always reduces token usage C. Default tool sets prevent models from reasoning D. Tailoring removes the need for coordinator oversight

Answer: A **Explanation:** Tool permissions should match the role-specific minimum needed for task completion. Broad defaults simplify setup but increase the chance of accidental misuse or scope creep.

Q11.

A coordinator wants a subagent to investigate a flaky UI test by reading logs and screenshots only. What is the best configuration?

A. Grant read access to the test artifacts and deny code-editing tools unless remediation is explicitly delegated later B. Grant repository write access because UI issues often lead to fixes C. Grant unrestricted browser automation and deployment access D. Provide no artifacts and let the subagent search everywhere

Answer: A **Explanation:** The subagent's current role is diagnosis from supplied artifacts, so its tools should reflect that limited scope. Broader privileges should be added only if the task later expands to remediation.

Q12.

Which prompt most effectively reinforces a restricted role for a subagent with limited tools?

A. "You can do anything useful." B. "Use only the provided artifacts to determine whether the regression is caused by localization formatting; do not infer unrelated repository issues." C. "Look broadly for improvements while you are there." D. "Escalate your permissions by default if the task feels difficult."

Answer: B **Explanation:** This prompt narrows both the objective and the permissible reasoning scope, matching the restricted tool environment. The other prompts encourage role drift or unnecessary escalation.

Q13.

Why should a coordinator avoid giving a subagent broader context than its role requires?

A. Extra context can leak irrelevant information and encourage work outside the intended boundary B. Broad context is always more accurate C. Context size has no effect on agent behavior D. It prevents the subagent from using structured data

Answer: A **Explanation:** Minimizing context to the task-relevant subset improves focus and reduces exposure to unnecessary data. More context is not automatically better, especially when it invites scope expansion.

Q14.

An architect is reviewing subagent setup quality. Which finding is most concerning?

A. A read-only analysis subagent received exactly the logs and dashboards it needed B. A patch-writing subagent was given access only to the target module and related tests C. A summary-only subagent was granted shell, edit, and network tools with no stated need D. A migration subagent received a system prompt defining completion criteria

Answer: C **Explanation:** This configuration violates the principle of restricting each subagent to its designated role and minimum required capabilities. The other scenarios show aligned permissions and scoped context.

Q15.

What is the best overall guideline for configuring subagent invocations under this objective?

A. Give every subagent the same powerful tool set to simplify orchestration B. Scope context to relevant artifacts, restrict tools to role-necessary capabilities, and use system prompts that explicitly reinforce the intended boundaries C. Rely on the model's judgment instead of technical restrictions D. Prefer under-scoping context so the subagent must ask for more details

Answer: B **Explanation:** This answer combines the three core levers named in the objective: tool restrictions, context scoping, and role-constraining system prompts. The other options either overexpose capability or deliberately create avoidable coordination overhead.